RQF LEVEL 3



GENCP302

NETWORKING AND INTERNET TECHNOLOGIES

C Programming Fundamentals

TRAINEE'S MANUAL

October, 2024

C PROGRAMMING FUNDAMENTALS

KOICA Korea International Cooperation Agency

TQUM TVET Quality Management Project

2024

AUTHOR'S NOTE PAGE (COPYRIGHT)

The competent development body of this manual is Rwanda TVET Board ©, reproduce with permission.

All rights reserved.

- This work has been produced initially with the Rwanda TVET Board with the support from KOICA through TQUM Project
- This work has copyright, but permission is given to all the Administrative and Academic Staff of the RTB and TVET Schools to make copies by photocopying or other duplicating processes for use at their own workplaces.
- This permission does not extend to making of copies for use outside the immediate environment for which they are made, nor making copies for hire or resale to third parties.
- The views expressed in this version of the work do not necessarily represent the views of RTB. The competent body does not give warranty nor accept any liability
- RTB owns the copyright to the trainee and trainer's manuals. Training providers may reproduce these training manuals in part or in full for training purposes only. Acknowledgment of RTB copyright must be included on any reproductions. Any other use of the manuals must be referred to the RTB.

© Rwanda TVET Board

Copies available from:

- *HQs: Rwanda TVET Board-RTB*
- *Web: www.rtb.gov.rw*
- **KIGALI-RWANDA**

Original published version: October 2024

ACKNOWLEDGEMENTS

The publisher would like to thank the following for their assistance in the elaboration of this training manual:

Rwanda TVET Board (RTB) extends its appreciation to all parties who contributed to the development of the trainer's and trainee's manuals for the TVET Certificate III in Networking and Internet Technologies, specifically for the module **"GENCP302: C Programming Fundamentals."**

We extend our gratitude to KOICA Rwanda for its contribution to the development of these training manuals and for its ongoing support of the TVET system in Rwanda

We extend our gratitude to the TQUM Project for its financial and technical support in the development of these training manuals.

We would also like to acknowledge the valuable contributions of all TVET trainers and industry practitioners in the development of this training manual.

The management of Rwanda TVET Board extends its appreciation to both its staff and the staff of the TQUM Project for their efforts in coordinating these activities.

This training manual was developed:

Under Rwanda TVET Board (RTB) guiding policies and directives



Under Financial and Technical support of



COORDINATION TEAM

RWAMASIRABO Aimable

MARIA Bernadette M. Ramos

MUTIJIMA Asher Emmanuel

PRODUCTION TEAM

Authoring and Review

UWAMAHORO Bonaventure

KWIZERA Jean Paul

Validation

MVUYEKURE Jean Damascene

MANISHIMWE Yves

Conception, Adaptation and Editorial works

HATEGEKIMANA Olivier

GANZA Jean Francois Regis

HARELIMANA Wilson

NZABIRINDA Aimable

DUKUZIMANA Therese

NIYONKURU Sylvestre

NYINAWUMUNTU Gaudence

Formatting, Graphics, Illustrations, and infographics

YEONWOO Choe

SUA Lim

SAEM Lee

SOYEON Kim

WONYEONG Jeong

KAYUMBA Fiston

Financial and Technical support

KOICA through TQUM Project

TABLE OF CONTENT

AUTHOR'S NOTE PAGE (COPYRIGHT) ---------- iii

ACKNOWLEDGEMENTS ---------- iv

TABLE OF CONTENT ---------- vii

ACRONYMS ---------- viii

INTRODUCTION ---------- 1

MODULE CODE AND TITLE: GENCP302 C PROGRAMMING FUNDAMENTALS ---------- 2

Learning Outcome 1: Describe Computer Programming Language ---------- 3

Key Competencies for Learning Outcome 1: Describe Computer Programming Languages 4

Indicative content 1.1: Identification of Programming Language 6

Indicative content 1.2: Development of a Flowchart 15

Indicative content 1.3: Development of an Algorithm 20

Learning outcome 1 end assessment 28

References 31

Learning Outcome 2: Write C Programming Codes ---------- 32

Key Competencies for Learning Outcome 2: Write C Programming Codes 33

Indicative content 2.1: Description of C Building Blocks 35

Indicative content 2.2: Description of C Program Structure 50

Indicative content 2.3: Applications of Condition Statements 69

Indicative content 2.4: Applications of Loops 81

Indicative content 2.5: Applications of Functions 95

Indicative content 2.6: Applications of Arrays 106

Indicative content 2.7: Solving of C Programming Problems 119

Learning outcome 2 end assessment 126

References 132

Learning Outcome 3: Perform Program Debugging ---------- 133

Key Competencies for Learning Outcome 3: Perform program debugging 134

Indicative content 3.1: Identification of Errors 136

Indicative content 3.2: Compilation of the C Program 143

Indicative content 3.3: Test of the C Program 148

Learning outcome 3 end assessment 158

References 162

ACRONYMS

ALGOL: Algorithmic Language

ALU: Arithmetic Logic Unit

API: Application Programming Interface

APL: A Programming Language

ASCII: American Standard Code for Information Interchange

BASIC: Beginners All Purpose Symbolic Instruction Code

COBOL: Common Business Oriented Language

CSS: Cascading Style Sheets

DLL: Dynamic Link Library

DMA: Direct Memory Access

EOF: End of File

FORTRAN: Formula Translation

GCC: GNU Compiler Collection

GPU: Graphics Processing Unit

GUI: Graphical User Interface

IDE: Integrated Development Environment

IO: Input/output

IOT: Internet of things

JIT: Just-In-Time compilation

JSON: JavaScript Object Notation

MVC: Model-View-Controller

OOP: Object-Oriented Programming

OS: Operating System

PL/I: Programming Language, Version 1

Prolong: Program in Logic

PSU: Power Supply Unit

RPG: Report Program Generator

INTRODUCTION

This trainee's manual includes all the knowledge and skills required in Networking and Internet Technologies specifically for the module of **"C Programming Fundamentals".** Trainees enrolled in this module will engage in practical activities designed to develop and enhance their competencies. The development of this training manual followed the Competency-Based Training and Assessment (CBT/A) approach, offering ample practical opportunities that mirror real-life situations.

The trainee's manual is organized into Learning Outcomes, which is broken down into indicative content that includes both theoretical and practical activities. It provides detailed information on the key competencies required for each learning outcome, along with the objectives to be achieved.

As a trainee, you will start by addressing questions related to the activities, which are designed to foster critical thinking and guide you towards practical applications in the labor market. The manual also provides essential information, including learning hours, required materials, and key tasks to complete throughout the learning process.

All activities included in this training manual are designed to facilitate both individual and group work. After completing the activities, you will conduct a formative assessment, referred to as the end learning outcome assessment. Ensure that you thoroughly review the key readings and the 'Points to Remember' section.

MODULE CODE AND TITLE: GENCP302 C PROGRAMMING FUNDAMENTALS

Learning Outcome 1: Describe Computer Programming Language
Learning Outcome 2: Write C Programming Codes
Learning Outcome 3: Perform Program Debugging

Learning Outcome 1: Describe Computer Programming Language



Indicative contents

1.1 Identification of programming Language

1.2 Development of a flowchart

1.3 Development of an algorithm

Key Competencies for Learning Outcome 1: Describe Computer Programming Languages

Knowledge	Skills	Attitudes
• Description of programming Languages. • Identification of programming IDE. • Description of flowchart. • Description of algorithm	• Installing programming IDE • Analysing a given problem • Arranging steps (logical sequence) according to the given problem • Designing flowchart • Developing an algorithm	• Being Patient • Have Critical thinking, and Attentive • Having Teamwork and creativity • Having Teamwork, and spirit



Duration:20 hrs



Learning outcome 1 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Describe correctly different types of programming languages as used in computer programming
2. Install properly C programming IDE according to the computer specifications.
3. Describe correctly different flowchart symbols according to their functions.
4. Design properly the flowchart according to the problem to be solved.
5. Develop properly the algorithms according to the problem to be solved.



Resources

Equipment	Tools	Materials
● Computers ● Projector ● Projection screen ● Printers	● Internet access ● C programming IDE	● Chalks ● Pens ● Books ● Papers ● Flip-chart

Indicative content 1.1: Identification of Programming Language

Duration: 5 hrs

Theoretical Activity 1.1.1: Identification of different types of computers programming

Tasks:

1: You are requested to answer the following questions related to the Identification of different types of computer programming languages.

I. What is a computer programming language?

II. What is a computer programming IDE?

III. Enumerate 2 role of programming IDE in telecommunications

IV. Give 2 types of programming languages.

2: Provide the answer for the asked questions and write them on papers.

3: Present the findings/answers to the whole class

4: Follow the expert view from the trainers

5: Ask questions for more clarification where necessary.

6: Read the **key readings 1.1.1.**

Key readings 1.1.1: Identification of different types of computers programming

Types of programming Languages

A computer programming language is a formal language comprising a set of instructions that produce various kinds of output. These languages are used to implement algorithms and to express computations in a format that can be understood and executed by computers.

- **Low-Level Languages**

  Low-level languages are programming languages that provide little or no abstraction from a computer's instruction set architecture. These languages are closely related to the hardware and are used to write programs that interact directly with the hardware.

✓ **Machine Language**: The most basic type of programming language, consisting of binary code (0s and 1s) that is directly understood by the computer's hardware.

✓ **Assembly Language**: A step above machine language, it uses symbolic names (mnemonics) to represent binary code. Requires an assembler to convert into machine language.

- **High-Level Languages**

  High-level programming languages are languages that are designed to be easy for humans to read, write, and understand. They provide a high degree of abstraction from the machine's hardware, allowing programmers to write code in a more natural and logical way, using words and symbols closer to human language.

✓ **Procedural Languages**: Focus on procedure or routine. Examples include C, Pascal, and Fortran.

✓ **Object-Oriented Languages**: Centered around objects and classes. Examples include C++, Java, and Python.

✓ **Functional Languages**: Based on mathematical functions. Examples include Haskell and Lisp.

✓ **Scripting Languages**: Used for automating tasks. Examples include Python, JavaScript, and Ruby.

✓ **Logic Programming Languages**: Based on formal logic. Example includes Prolog.

✓ **Markup Languages**
Used for annotating a document in a way that is syntactically distinguishable from the text. Examples include HTML, XML, and LaTeX.

✓ **Domain-Specific Languages**
Tailored to specific application domains. Examples include SQL for databases and MATLAB for mathematical computations.

- **Role of Programming Languages in Electronics and Telecommunication**

✓ **Embedded Systems**: Programming languages like C, C++, and Assembly are used to develop firmware and software for microcontrollers and processors in embedded systems.

✓ **Signal Processing**: Languages like MATLAB and Python (with libraries like NumPy and SciPy) are used for signal processing and analysis.

✓ **Network Simulation and Protocol Development**: Languages like C, C++, and

Python are used to develop network protocols and simulate telecommunication networks.

- ✓ **Automation and Control Systems**: Used in designing automated and control systems in telecommunications infrastructure.
- ✓ **IoT Development**: Programming languages such as C, Python, and JavaScript are used for developing IoT applications and devices.
- ✓ **DSP (Digital Signal Processing)**: Specialized languages and libraries are used to implement DSP algorithms for communication systems.

Types of C Programming IDEs:

A computer programming Integrated Development Environment (IDE) is a software application that provides comprehensive facilities to computer programmers for software development. An IDE typically consists of a source code editor, build automation tools, and a debugger.

- **Cross-Platform IDEs**

- ✓ **Eclipse**: Widely used, supports multiple languages, extensible via plugins.
- ✓ **Code:Blocks**: Open-source, highly configurable, and supports multiple compilers.
- ✓ **NetBeans**: Modular IDE with support for C/C++ development.

- **Windows-Specific IDEs**

- ✓ **Microsoft Visual Studio**: Powerful IDE with extensive features for debugging and development.
- ✓ **Dev-C++**: Lightweight and simple, suitable for beginners.

- **Linux-Specific IDEs**

- ✓ **CodeLite**: Open-source IDE designed for C/C++ development.
- ✓ **KDevelop**: Integrated development environment for Linux users, supports various programming languages.

- **MacOS-Specific IDEs**

- ✓ **Xcode**: Apple's official IDE for macOS development, supports C/C++ among other languages.

- **Online IDEs**

- ✓ **Repl.it**: Web-based IDE supporting many languages, including C.
- ✓ **JDoodle**: Online compiler and IDE for C/C++ programming



Practical Activity 1.1.2: Installation of C programming IDE



Task:

1. Referring to previous activity, you are requested to perform the given task.

John has a computer which doesn't have C Programming IDE and he needs to write a program which will display" TVET SCHOOL". You are asked to install C Programming IDE in that computer.

2. Using key readings **1.1.2**, List out procedures to be used to perform the given tasks

3. Install the c Programming IDE according the previous procedures in task 2.

4. Present the installed IDE to the whole class.

5 Ask clarification where necessary.

6. Read key reading **1.1.2**

7. Perform the task provided in application of learning **1.1.**



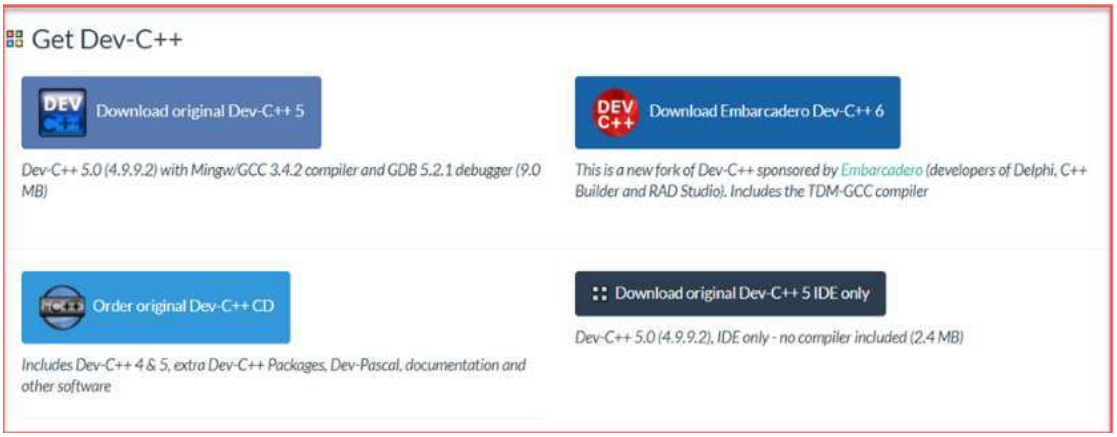
Key readings 1.1.2: Installation of C programming IDE

Installation of c programming IDE

- How to download setup .exe

There are many compilers available for C. You need to download anyone. Here, we are going to use Dev-c++. It will work for both C and C++. To install the Turbo C software, you need to follow the following steps.

- ✓ Step 1) First you must download the Dev C++ on your Windows machine. Visit to Download Dev C++: http://www.bloodshed.net/
- ✓ Step 2) There are packages for different Operating Systems.



- ✓ Step 3) Under package Dev-C++ 5.0 (4.9.9.2) with Mingw/GCC 3.4.2 compiler and GDB 5.2.1 debugger (9.0 MB) Click on the link "Download from SourceForge".



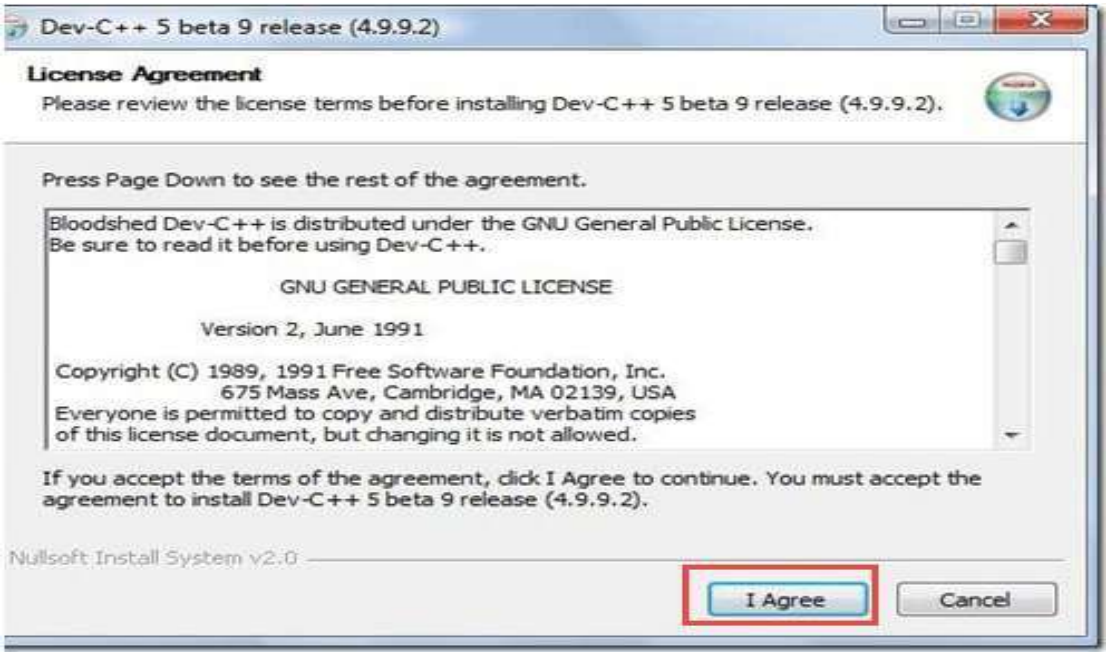
- ✓ Step 4) This package will download C++ .exe file for Windows that can be used to install on Windows 7/8/XP/Vista/10.
- ✓ Step 5) You will direct to SourceForge website, and your C++ download will start automatically.

Click on save button to save. By default, it is saved in the "Downloads" folder.

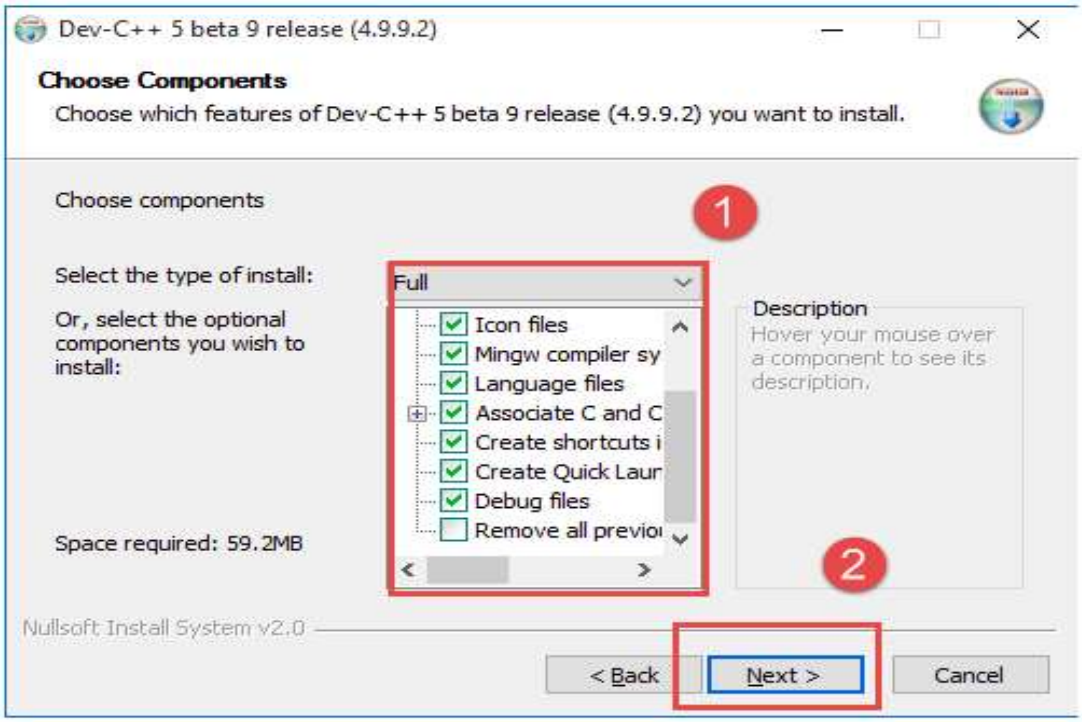
- How to install setup.exe
- ✓ Step 1) Locate setup.exe file of c programming IDE
- ✓ Step 2) Double-click the .exe file. A dialog box will appear then click ok.



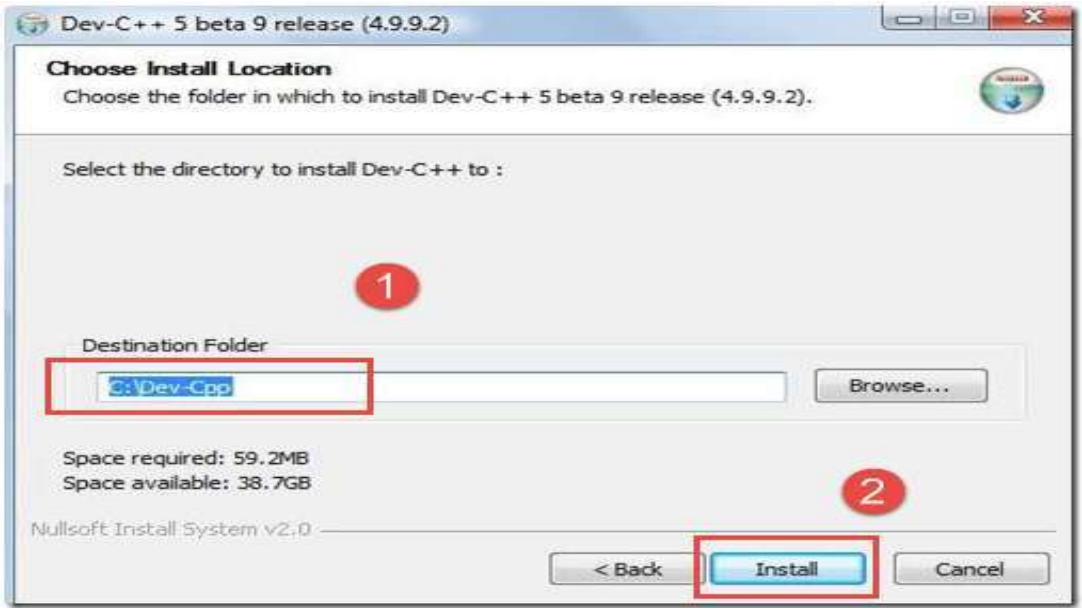
✓ Step 3) Then a screen for license agreement will appear. Click on “I agree” to proceed further.



✓ Step 4) Just click on the “next” button.



✓ Step 5) Click on the “Install” button.



Now, Dev C++ is installed successfully on your Windows. Select ” Run Dev C++” to run it and click on the ” Finish” button.



Setting Up C Programming Environment

- **Installing a Compiler**

✓ **GCC (GNU Compiler Collection)**: Available for Linux, Windows (via MinGW), and macOS.

✓ **Clang**: A compiler for C language family, available for Linux, Windows, and macOS.

- **Choosing and Installing an IDE**

✓ Select an IDE based on your operating system and preferences (e.g., Visual Studio for Windows, Eclipse for cross-platform use).

✓ Download and install the chosen IDE from its official website.

- **Configuring the IDE**

✓ **Setting Compiler Path**: Ensure the IDE knows the path to the installed compiler.

✓ **Configuring Build and Debug Settings**: Set up the build configuration (debug/release), link libraries, and include directories.

✓ **Writing and Running Code**: Create a new project, write C code, and use the IDE's build and run functionalities to compile and execute the code.

- **Using Command Line Tools**

✓ **Compiling via Command Line**: Use commands like gccfilename.c -o outputfile to compile C programs.

✓ **Running the Program**: Execute the compiled binary from the command line using ./outputfile.

- **Libraries and Dependencies**

✓ **Standard Libraries**: Include standard libraries like stdio.h, stdlib.h, etc., in your programs.

✓ **Third-Party Libraries**: Link any required third-party libraries during compilation.

- **Version Control**

✓ **Using Git**: Initialize a Git repository, commit your code, and use platforms like GitHub or GitLab for version control and collaboration.



Points to Remember

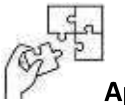
Types of programming Languages

- A computer programming language is a formal language used to write instructions that a computer can execute.
- There are two types: **low-level languages**, which are closely tied to hardware, and **high-level languages**, which offer more abstraction.

- Programming languages are essential in telecommunications for developing firmware and software for embedded systems, network monitoring, and automation.
- C Programming is a widely-used, easy-to-learn general-purpose language that helps understand core programming concepts, making it easier to learn other high-level languages.
- Integrated Development Environments (IDEs) are tools where programmers write code; C has various IDEs compatible with different operating systems like Windows, macOS, and Linux, with some being cross-platform.

Setup the environment if C programming IDE

- Visit the official website of the IDE (e.g., Visual Studio, Code::Blocks, Eclipse) and download the appropriate installer or Locate the setup of C programming IDE .
- Install a C compiler (e.g., GCC for Linux/Mac, MinGW for Windows) if the IDE doesn't include one.
- Follow the installation instructions provided by the installer, adjusting settings as needed.
- Set up the IDE to use the correct compiler and debugger.
- Configure paths for libraries and include files if necessary.
- Create a simple C program to ensure everything is working correctly.
- Compile and run the program to get the output.



Application of learning 1.1.

Madam Rose needs to write a C program to display her name, but her computer lacks a C programming IDE. you are requested install a C programming IDE on her computer.

Indicative content 1.2: Development of a Flowchart

Duration: 5 hrs

Theoretical Activity 1.2.1: Description of the flowchart

Tasks:

1: In small groups, you are requested to answer the following question related to the description of the flowchart symbols.

I. What is a flowchart?

II. What are the importances of a flowchart in visualizing processes.

III. What are symbols used to design a flowchart

2: Provide the answer for the asked questions and write them on papers.

3: Present the findings/answers to the whole class

4: In addition, ask questions where necessary.

5: For more clarification, read the key **readings 1.2.1**

Key readings 1.2.1. Description of the flowchart

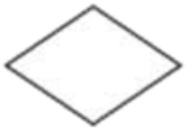
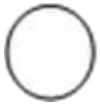
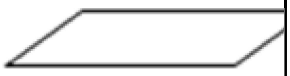
Description of Flowchart

- **Definition of Flowchart**

  Flowchart is a graphical representation of the sequence of operations in an information system or program. Information system flowcharts show how data flows from source documents through the computer to final distribution to users. Program flowcharts show the sequence of instructions in a single program or subroutine. Different symbols are used to draw each type of flowchart.

  Flowchart symbols

Name of the symbol	Diagram	Description
Oval		Denotes the start and end of the

		flowchart
Rectangle		A calculation or assigning of value to a variable.eg.Addition,subtraction,division,etc
Rhombus or Diamond		Denotes a decision (branch)to be made.The program should continue along one of two routines.True of False
Circle		Can be used to eliminate lengthy of flowrines. It's used to indicate that one symbol is connected to another.
Parallelogram		Denotes an Input operations
Flow Line		Lines indicate the sequence of steps and the direction of flow. The flow is assumed to go from top to bottom and from left to right.



Practical Activity 1.2.2: Designing a flowchart



Task:

1: Referring to previous activities (1.2.1) you are requested to perform the given task.

Using a flowchart Describe the steps involved in going to the cinema to watch a movie, including actions like choosing a film, buying tickets, and making decisions, such as verifying showtime availability.

2: From key readings 1.2.2, list out procedures to be used to perform the given task f

3: Perform the given tasks according to the procedures listed in task 2.

4: Present your work to the whole class.

5: Ask clarification where necessary



Key readings 1.2.2: Designing a flowchart

Design a flowchart

A flowchart is a visual representation of a process, algorithm, or system that outlines the steps or actions involved in a sequence.

- **Develop a flowchart.**

steps to develop a flowchart according to the given problem

Step 1: Know the purpose of your flowchart.

Before knowing how to draw a flowchart, it is mandatory to find out why you're creating one in the first place. Your flowchart can have various goals, such as:

✓ Making a complex process easier to understand

✓ Improving an existing business process by identifying bottlenecks

✓ Explaining or communicating a process to someone else

✓ Standardizing a process for consistency and efficiency

When you know the purpose of your flowchart, it's easier to choose the type of flowchart to create, pick the most relevant template, and know whether to focus more on design or function when making one.

Step 2: Start with a template.

Creating a flowchart from scratch can be intimidating. Luckily, you don't have to. There are plenty of flowchart templates available online that you can customize and adapt for your own use. Try to select a template that's created specifically for your field or industry so it takes you less time and effort to finalize.

Step 3: Add shapes and symbols.

Shapes and symbols play a vital role in any flowchart. They define the components of your flowchart, and each symbol represents something unique. After you've identified the purpose of your flowchart, it's time to draw your diagram. The first thing you need to do here is add the symbols that represent all the components in your flowchart.

Step 4: Connect your shapes with lines and arrows.

Once you've added all the different shapes you need, it's time to connect each one with lines and arrows. We recommend using arrows if you want to show a specific direction of your flow of information. For example, the flowchart example below visualizes how information is flowing in different directions with the help of arrows.

Step 5: Split paths or add decisions.

In some cases, you may want to split paths and add decisions to your flowchart, such as adding two different paths based on Yes/No decisions. This is useful for visualizing complex processes that require people to take different actions based on varying inputs and outputs.

Step 6: Customize your flowchart's appearance.

The next step is to customize your flowchart's appearance. Your flowchart's design will depend on how you plan to use your diagram, where you'll use it and who the audience is. First, you may want to change the size, color, border, roundness and other characteristics of your shapes, symbols and lines.

Step 7: Download or share your flowchart.

Once your flowchart is ready, it's time to download it or share it with your target audience.

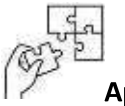


Points to Remember

- Flowcharts are diagrammatic representations of processes.
- Flowchart Symbols ovals, rectangles, diamonds, and arrows to show the sequence of steps, decision points, and outcomes. They simplify complex processes, enhance communication, and aid in problem-solving, documentation, and training.

Design a flowchart

- Define the Objective
- Use Standard Symbols
- Maintain Logical Flow



Application of learning 1.2.

In southern province different sectors want to construct the playground for each school. They need calculate the area of each ground using its length and width. Draw a flowchart that will accept the length, width to calculate and print the area of the playground. (area = length * width).

Indicative content 1.3: Development of an Algorithm

Duration: 10 hrs

Theoretical Activity 1.3.1: Description ofalgorithms.

Tasks:

1: In small groups, you are requested to answer the following questions related to the description of algorithms.

I. What is an algorithm?
II. What is pseudo code, and how is it used in the algorithm design process?
III. Enumerates the different types of algorithms.

2: Provide the answer for the asked questions and write them on papers/flipchart or white board.

3: Present the findings/answers to the whole class

5: Ask for more clarification, where necessary.

4: For more clarification, read the **key readings 1.3.1**

Key readings 1.3.1.: Description ofalgorithms.

- **Definition of an algorithm**
  Algorithm is the sequence of instructions which are involved in solving a given problem or accomplishment of a given task. It can be translated into a programming language in order to produce the result.A set of steps that generates a finite sequence of elementary computational operations leading to the solution of a given problem is called an algorithm.

- **Structure of an algorithm**

  An algorithm is made mainly of the following parts:

✓ The variable declaration line
✓ The beginning of an algorithm
✓ The instruction's part
✓ The end

Example:

Var A as Integer

Start

A←5

End

Explanations:

Var A as Integer is the variable declaration line

Start marks the beginning of an algorithm

A←5 the instructions part

End marks the end of an algorithm

- **Pseudo code**
  Artificial and informal language that helps programmers develops algorithms. Pseudo code is very similar to everyday English.
  Pseudo code is a "text-based" detail (algorithmic) design tool. The rules of Pseudo code are reasonably straightforward. All statements showing "dependency" are to be indented. These include while, do, for, if, switch.

Types of algorithms

Algorithms are step-by-step procedures or formulas for solving problems, executing tasks, or achieving objectives. They can be classified into various types based on their design, structure, and purpose.

- **Sequential Algorithms**:

✓ Execute instructions in a linear sequence from start to finish.
✓ Example: Simple arithmetic operations.

- **Iterative Algorithms**:

✓ Repeat a sequence of instructions until a specific condition is met.
✓ Example: Searching or sorting algorithms like Bubble Sort or Binary Search.

- **Recursive Algorithms**:

✓ Call themselves with smaller instances of the same problem until reaching a base case.
✓ Example: Factorial calculation or Fibonacci sequence.

- **Divide and Conquer Algorithms**:

- ✓ Break down a problem into smaller, manageable sub-problems.
- ✓ Solve each sub-problem independently and combine the solutions.
- ✓ Example: Merge Sort or Quick Sort.

- **Greedy Algorithms**:

- ✓ Make locally optimal choices at each stage with the hope of finding a global optimum.
- ✓ Example: Minimum Spanning Tree algorithms like Prim's or Kruskal's.

- **Dynamic Programming Algorithms**:

- ✓ Solve complex problems by breaking them down into simpler sub-problems.
- ✓ Store solutions to sub-problems to avoid redundant calculations.
- ✓ Example: Fibonacci sequence using memorization.

- **Backtracking Algorithms**:

- ✓ Attempt to build a solution incrementally and backtrack when reaching a dead end.
- ✓ Example: N-Queens problem or Sudoku solving.

- **Randomized Algorithms**:

- ✓ Introduce randomness or probability in their execution.
- ✓ Example: Randomized Quick Sort or Monte Carlo algorithms.

- **Heuristic Algorithms**:

- ✓ Produce good solutions to problems that cannot be solved optimally.
- ✓ Often used in decision-making or optimization problems.
- ✓ Example: Genetic algorithms or Ant Colony Optimization.

- **Parallel Algorithms**:

- ✓ Execute multiple operations simultaneously to reduce execution time.
- ✓ Suitable for tasks that can be divided into independent sub-tasks.
- ✓ Example: Parallel Matrix Multiplication.

- **Importance of Algorithm Classification**

- ✓ **Problem-Specific Efficiency**: Different algorithms are suited to different types of

problems based on their characteristics and constraints.

- ✓ **Optimization Goals**: Understanding algorithm types helps in choosing the most efficient solution approach.
- ✓ **Algorithm Design**: Helps in designing new algorithms or optimizing existing ones for specific tasks.

Problem Solving: Provides a structured approach to solving computational problems across various domains.



Practical Activity 1.3.2: Development of an algorithm



Task:

1: Refer to previous activity 1.3.1, you are requested to perform the given task.

You are tasked with developing an algorithm and pseudocode to sort books on a library bookshelf by their titles alphabetically. The library currently has a disorganized shelf where books are placed randomly, and the librarian wants them arranged in alphabetical order for easier access and browsing by library patrons.

2: By using **key readings 1.3.2**, list out procedures to be used to develop an algorithm.

3: Write an algorithm using the procedures list above.

4: Present your work to whole class.

5: Ask clarification where necessary



Key readings 1.3.2: Development of an algorithm

Development of an algorithm

An **algorithm** is a step-by-step procedure or set of instructions to solve a specific problem or perform a specific task. When developing an algorithm, you need to break down the problem into smaller, manageable steps that lead to the desired solution.

- **Understanding the Problem:**

- ✓ **Define the Problem:** Clearly understand and define the problem you are trying to solve. Gather all necessary information and requirements.
- ✓ **Identify Inputs and Outputs:** Determine what inputs are needed and what outputs are expected. This helps in designing the steps required to transform inputs into outputs.

- **Breaking Down the Problem:**

- ✓ **Decomposition:** Break down the problem into smaller, manageable parts. Each part should be easier to handle and solve.
- ✓ **Modularity:** Design the algorithm in a modular way, where each module or function handles a specific part of the problem.

- **Designing the Algorithm:**

- ✓ **Step-by-Step Instructions:** Outline the steps needed to solve the problem in a logical sequence. Each step should be clear and unambiguous.
- ✓ **Flow Control:** Determine the control structures needed, such as loops, conditionals, and function calls, to manage the flow of the algorithm.

- **Writing Pseudocode:**

- ✓ **Informal Language:** Use pseudocode, which is a mixture of natural language and programming concepts, to outline the algorithm. Pseudocode helps in planning the algorithm without worrying about syntax.
- ✓ **Clarity:** Ensure the pseudocode is easy to understand. It should be simple enough for someone without programming knowledge to follow the logic.

- **Analyzing the Algorithm:**

- ✓ **Efficiency:** Analyze the algorithm for time and space complexity. Consider how the algorithm scales with increasing input size.
- ✓ **Correctness:** Verify that the algorithm correctly solves the problem for all possible inputs. Consider edge cases and test the algorithm with various scenarios.

Examples

1. **Formulate an Algorithm to Check if a Number is Prime.**

- **Problem Understanding:**
  - Input: A number n.
  - Output: True if n is prime, False otherwise.

- **Steps:**
  - If n <= 1, return False.
  - Loop from 2 to the square root of n.
  - If n is divisible by any number in this range, return False.
  - If no divisor is found, return True.
- **Pseudocode:**

Algorithm IsPrime(n)

1. If n <= 1 then

   a. Return False

2. For i from 2 to sqrt(n):

   a. If n % i == 0 then

i. Return False

3. End loop

4. Return True

End Algorithm

2. You are given a list of numbers. Develop an algorithm to find the largest number in the list.

Step-by-Step Development of the Algorithm:

1. **Understand the Problem:**

   - ✓ We need to find the largest number from a list of numbers.
   - ✓ Input: A list of n numbers (e.g., [3, 5, 7, 2, 8]).
   - ✓ Output: The largest number in the list (e.g., 8).

2. **Identify Key Operations:**

   - ✓ Iterate through the list of numbers.
   - ✓ Compare each number to find the largest one.

3. **Outline the Steps:**

Step 1: Initialize a variable max to store the largest number. Start by setting it to the first number in the list.

Step 2: Loop through the list starting from the second number.

Step 3: For each number, compare it with max. If the number is greater than max, update max with this number.

Step 4: Continue the loop until all numbers have been checked.

Step 5: The value in max at the end of the loop is the largest number.

Step 6: Output the value of max.

4. **Translate into Pseudocode:**

Algorithm FindLargestNumber(List)

1. Set max = List[0]
2. For each number in List starting from the second number:
   a. If number > max then

i. Set max = number

3. End loop
4. Output max

End Algorithm



Points to Remember

An algorithm is a sequence of instructions designed to solve a problem or achieve a specific task. It can be translated into a programming language to produce a desired result.

Structure of an Algorithm:

- Variable Declaration: Defines variables used in the algorithm.
- Beginning: Marks the start of the algorithm.
- Instructions: Contains the sequence of steps to be executed.
- End: Marks the completion of the algorithm.
- Types of Algorithms: Sequence, Branching (Selection), Loop (Repetition):
- **Pseudo Code:**

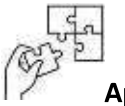
- ✓ An informal, text-based language for designing algorithms.
- ✓ Helps programmers plan and outline solutions before coding.
- ✓ Rules are straightforward and aid in algorithmic design.

- **Develop an algorithm**

✓ Obtain detailed information on the problem.
✓ Analyse the problem.
✓ Think of a problem-solving approach.
✓ Review the problem-solving approach and try to think of a better Alternative.
✓ Develop a basic structure of the Algorithm.
✓ Optimize, Improve and refine.

- **Write the pseudo code**

✓ Always capitalise the initial word (often one of the main six constructs).
✓ Make only one statement per line.
✓ Indent to show hierarchy, improve readability, and show nested constructs.
✓ Always end multi-line sections using any of the END keywords (ENDIF, ENDWHILE, etc.).
✓ Keep your statements programming language independent?
✓ Use the naming domain of the problem, not that of the implementation. For instance: “Append the last name to the first name” instead of “name = first+ last.”
✓ Keep it simple, concise and readable.



Application of learning 1.3.

The GB company is located in Kigali; they pay their employees using paper. Due to this issue the employees did get easily their payroll statement. As a programmer Write an algorithm and pseudocode to calculate and display the total monthly expenses based on various categories like rent, groceries, utilities, and entertainment. The user should input expenses for each category.

Learning outcome 1 end assessment

Theoretical assessment

1. What is a symbol in a flowchart represents the start and end of the process?
2. What is a programming language?
3. Explain the purpose of a flowchart in software development.
4. Describe the steps you would follow to install a C programming IDE like Code::Blocks on your computer.
5. Compare and contrast the use of procedural programming and object-oriented programming paradigms in software development.
6. Evaluate the pros and cons of using an interpreted language like Python for developing a web application.
7. Name two programming languages commonly used for embedded systems development.
8. What is the primary function of pseudo code in the development process?
9. Given a scenario where you need to automate a repetitive task in a telecommunications system, which programming language would be most appropriate and why?
10. Analyze the use of flowcharts versus pseudo code in a team setting for a large-scale project. Which would you prefer and why?
11. Copy and complete the following table

Name of the symbol	**Diagram**	**Description**
Oval	……	……
……	……	It is used for data processing
……		……
Circle	……	……
……		……

12. Compare and contrast different programming paradigms (procedural, object-oriented, functional, and logic programming). Provide examples of languages that exemplify each paradigm and discuss their primary use cases.
13. Describe the differences between compiled and interpreted programming languages. Discuss the advantages and disadvantages of each approach with examples of languages that use these methods.

Multiple choice questions

14. Which of the following is an advantage of compiled languages over interpreted languages?
    a) Easier to debug
    b) Faster execution
    c) Platform independence
    d) Immediate feedback

15. Which of the following steps would be the best starting point for creating a new software application?
    a) Writing the final code
    b) Designing a flowchart or pseudo code
    c) Debugging the code
    d) Compiling the program

16. In what scenario would functional programming be preferred over logic programming?
    a) Concurrent processing tasks
    b) Artificial intelligence applications
    c) Designing embedded systems
    d) Web development

17. Which IDE would you choose for C/C++ development if you needed a lightweight, customizable environment?
    a) Eclipse
    b) Visual Studio
    c) Code::Blocks
    d) CLion

18. Pseudo code is primarily used for:
    a) Debugging code
    b) Writing final programs
    c) Describing the steps in an algorithm
    d) Designing hardware

Practical assessment

1. You need to write an algorithm and pseudocode for creating a grocery shopping list. The program should allow the user to input items they need to buy, and then display the complete list of items.
2. Develop an algorithm and pseudocode for a library management system to add and display books. The user can enter the book's title and author, and the system should store and display the list of books.
3. Create an algorithm and pseudocode for a simple calculator that performs addition, subtraction, multiplication, and division based on user inputs.

END



References

A. Books

Graham, D. L. (2019). *C Programming Language: A Step by Step Beginner's Guide to Learn C Programming* . Amazon.

King, K. (2008). *C Programming: A Modern Approach 2nd Edition.* Amazon.

McGrath, M. (2018). *C Programming in easy steps, 5th Edition.* Amazon.

Miller, G. P. (2013). *C Programming Absolute Beginner's Guide.* Amazon.

Norton, W. (2021). *he C Programming Language (Simplified for Beginners).* Amazon.

Thareja, R. (2011). *Data Structures Using C.* Amazon.

B. Websites

https://www.scribd.com/presentation/413537916/Lect1-Algorithms-and-Flowchart-ppt?utm_medium=cpc&utm_source=google_pmax&utm_campaign=3Q_Google_Performance-Max_RoW&utm_term=&utm_device=c&gclid=CjwKCAjw36GjBhAkEiwAKwIWyYBOAwSwzUDmki-xCos35zYEx-s7uQqw4Q2rHSpf2FG2VySd7fYxMRoCmJAQAvD_BwE#

https://faradars.org/wp-content/uploads/2015/07/algorithm-and-flow-chart.pdf

https://terrorgum.com/tfox/books/flowchartandalgorithmbasics.pdf

https://www.w3resource.com/c-programming-exercises/conditional-statement/index.php


evaluated
statement
standard
case
void
contains
array
constant
variables
function
POINTER
size
zero
define
int
read
char
USED
CHARACTER
STRUCT
input
error
operator
double
structure
time
write
String
specifier
SPACE
library
object
expression
declaration
RESULT
external
NULL
INITIALIZED
PROGRAM
include
TYPE
return
CALLED

Indicative contents

2.1. Description of C programming building blocks

2.2. Description of C program structure

2.3. Application of condition statements

2.4. Application of loops

2.5. Application of functions

2.6. Application of Arrays

2.7. Solving a C programming problem

Key Competencies for Learning Outcome 2: Write C Programming Codes

Knowledge	Skills	Attitudes
• Description of C structure • Description of control structure • Differentiating interpreter from compiler • Differentiate statement from expression • Differentiation variable from constants • Differentiation different data types • Description of functions • Identification of the scope of a variable	• Installing C based IDE • Applying variables • Using Operators • Using different data types. • Applying conditional statements. • Applying functions • Applying control structure • Applying Loops • Applying Arrays • Document program • Using comments	• Having Critical thinking • Being Passionate • Having Team work spirit • Have Persistence • Being Hard working • Being Self-motivated • Having Self confidence



Duration: 50 hrs



Learning outcome 2 objectives:

By the end of the learning outcome, the trainees will be able to:

1. Describe correctly the building blocks as per C programming language
2. Describe correctly the C program structure as per functional program
3. Apply properly a condition statements based on a syntax and certain conditions.
4. Apply properly C loops based on syntax and certain instructions.
5. Apply properly C functions based on syntax and specified task.
6. Apply properly C Arrays based on syntax, data store and their use.
7. Solve a real-world problem in line with C programming language.

 **Resources**

Equipment	Tools	Materials
● Computer ● Projector	● Visual Studio Code ● Dev-C++ ● Python tutor code visualizer ● Internet	● Electricity

Indicative content 2.1: Description of C Building Blocks

Duration: 5 hrs

Theoretical Activity 2.1.1: Description of C Tokens

Tasks:

1: In small groups, you are requested to answer the following questions related to the description of resonant circuits.

I. What is a token in the context of the C programming language?

II. How are tokens used to represent fundamental building blocks in C code?

III. Name and describe the five main categories of C tokens.

IV. What are identifiers in C, and how do they differ from keywords?

V. Give examples of valid and invalid identifiers in C.

VI. Explain the role of keywords in C and provide some examples.

VII. Name the three primary types of constant tokens in C.

2: Provide the answer for the asked questions and write them on papers.

3: Present the findings/answers to the whole class

4: For more clarification, read the key readings 2.1.1.

5: In addition, ask questions where necessary.

Key readings 2.1.1: Description of C Tokens

C Tokens

In C programming, tokens are the smallest units of a program that have meaningful interpretation by the compiler. These tokens form the basic building blocks of C syntax. Here are the main types of tokens in C:

- **Keywords:**
  - ✓ Keywords are reserved words that have predefined meanings in C. They cannot be used as identifiers (variable names, function names, etc.).
  - ✓ Examples: int, float, if, else, for, while, return, void, switch, case, break, continue, sizeof, typedef, struct, union, enum, const, static, extern, volatile, auto, register, default, do, double, short, long, signed, unsigned, char, goto.

- **Identifiers:**
  - ✓ Identifiers are names given to variables, functions, arrays, structures, etc., by the programmer.
  - ✓ Rules for identifiers:
    - Must start with a letter (A-Z or a-z) or an underscore (_).
    - Can be followed by letters, digits (0-9), or underscores.
    - Case-sensitive.
    - Cannot be a keyword.
  - ✓ Example: main, sum, counter, MAX_SIZE, is_valid.
- **Constants:**
  - ✓ Constants are fixed values that do not change during program execution.
  - ✓ Types of constants:
    - ✓ **Integer Constants:** Whole numbers without decimal points.
      - Example: 123, -456, 0, 0xFF (hexadecimal).
    - ✓ **Floating-point Constants:** Numbers with decimal points.
      - Example: 3.14, -0.5, 2.0e3 (exponential).
    - ✓ **Character Constants:** Single characters enclosed in single quotes.
      - Example: 'A', '7', '\n' (newline character).
    - ✓ **String Literals:** Sequences of characters enclosed in double quotes.
      - Example: "Hello, World!", "C programming".
- **String and Character Literals:**
  - ✓ String literals are sequences of characters enclosed in double quotes (").
  - ✓ Character literals are single characters enclosed in single quotes (').
  - ✓ Example: "Hello", 'A', "123", '\n'.
- **Operators:**
  - ✓ Operators perform operations on variables and values.
  - ✓ Types of operators:
    - Arithmetic Operators: +, -, *, /, %
    - Relational Operators: ==,!=, <, >, <=, >=
    - Logical Operators: &&, ||,!
    - Bitwise Operators: &, |, ^, ~, <<, >>
    - Assignment Operators: =, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=
    - Increment and Decrement Operators: ++, --
    - Conditional (Ternary) Operator: ? :
    - Address and Indirection Operators: &, *
  - ✓ Example: a + b, x < y, !flag, ~mask, a = b, x && y
- **Punctuation Symbols:**
  - ✓ Punctuation symbols are special characters that serve as separators or delimiters in C.
  - ✓ Examples: , (comma), ; (semicolon), : (colon), . (dot), () (parentheses), {}

(braces), [] (brackets).

- **Whitespace:**
  - ✓ Whitespace includes spaces, tabs, and newline characters.
  - ✓ Whitespace separates tokens but is otherwise ignored by the compiler.
- **Comments:**
  - ✓ Comments provide information to the programmer and are ignored by the compiler.
  - ✓ Types of comments:
    - **Single-line comments:** // this is a comment.
    - **Multi-line comments:** /* this is a multi-line comment */.



Practical Activity 2.1.2: Writing the Data types in a c program



Task:

1. Read the following task, you are requested to perform the given task.
John, a computer science student, is tasked with developing a program that manages inventory for a small retail store. His program needs to handle different types of data, from item quantities to prices, and ensure accurate calculations and storage of information. Write all data types John need to complete his task.

2. Follow the instructions in key readings 2.1.2 related to the given task

3. Write down the findings from the discussions.

4. Present the finding to whole class.

5. Ask clarification where necessary



Key readings 2.1.2: : Writing the Data types in a c program

Data type in c programming

In C programming, data types are fundamental for defining the type of data that variables can hold and manipulate. Implementing data types effectively ensures proper storage, manipulation, and representation of data within a program. Here's an overview of how data types are implemented in C programs:

- **Choosing Appropriate Data Types:**